CHARACTER ATTACK SETUP

Set up an animated player, create a standalone attack Action Clip, call it from a gameplay script, and prevent button-spam or animation restarts.

1. Character	2. Action Clip	3. Script	4. Play
Skeletal model and locomotion	One-shot attack asset	Input, lock and cooldown	Verify one attack per press

Result
Left mouse or F starts the attack only if the previous action has finished and the recovery timer is clear. Holding the button does not repeatedly fire it.

Applies to the current 3DG Engine workflow

Character Editor - Animation Graph - Clip Editor - Content assets - Editor-created C++ scripts - Compile Scripts & Restart

1. Before You Start

Prepare one skeletal character and at least one attack animation. The attack may be inside the character FBX or in a separate FBX. Separate animation files must use the same skeleton and bone names as the character.

Asset	Purpose	Example
Skeletal model	Character mesh and skeleton	WizardSM.fbx
Locomotion clips	Idle, walk, run and optional jump	Idle.fbx, Run.fbx
Attack clip	One-shot attack animation	StaffAttack.fbx
Character asset	Mesh, graph, controller, collider and actions	Wizard.3dgcharacter
Action asset	Saved script-callable attack definition	StaffAttack.3dgclip

Recommended Content layout

```
Content/
  Assets/
    Characters/Wizard.3dgcharacter
    Animations/WizardLocomotion.3dggraph
    Animations/StaffAttack.3dgclip
    Meshes/WizardSM.fbx
    AnimationClips/StaffAttack.fbx
  Scripts/PlayerAttack.h
```

Create the base character

1. **Open Character Editor**
   Create a new character asset and choose the skeletal model.
2. **Correct model orientation**
   Use the render-only model rotation so the mesh stands upright and faces the forward guide.
3. **Configure collision**
   Use a capsule collider sized around the body. Keep the capsule upright and centered.
4. **Enable player movement**
   Add Player Controller and tune movement, turn speed and camera mode.
5. **Assign locomotion**
   Choose the saved Animation Graph containing idle, walk, run and jump logic.

Keep responsibilities separate
The Animation Graph owns continuous locomotion states. The attack is a standalone Action Clip and is started by gameplay code.

2. Create the Attack Action Clip

Open **Clip Editor**. The Clip Editor packages an animation source and playback settings into a reusable **.3dgclip** asset. It does not add an attack state to the locomotion graph.

1. **Choose the source file**
   Select StaffAttack.fbx, or the FBX that contains the attack take.
2. **Choose the internal clip**
   Pick the exact take from the source. Generic imported names such as Unreal Take are valid.
3. **Set the Action Name**
   Use a clear unique name such as StaffAttack. This is the name used by the script.
4. **Enable Action Clip**
   This forces one-shot playback and disables looping.
5. **Choose body coverage**
   Full Body locks movement. A bone mask, such as Spine, layers the attack over locomotion.
6. **Tune playback**
   Start with Fade In 0.06, Fade Out 0.15, Speed 1.0 and Strip Root Motion enabled.
7. **Save in Content**
   Save as Content/Assets/Animations/StaffAttack.3dgclip.

Setting	Recommended start	Effect
Action Clip	On	Makes the clip a one-shot scripted action.
Loop	Off (automatic)	Prevents an attack from repeating forever.
Strip Root Motion	On	Keeps the capsule authoritative and prevents drifting.
Body Mask	Full Body	Stops movement until the attack completes.
Fade In / Out	0.06 / 0.15 sec	Softens the transition into and out of the action.
Speed	1.0	Animation playback rate.

Full Body versus masked
Full Body: movement is locked while the action plays, which suits committed melee attacks. **Masked:** choose a root bone such as Spine to keep the legs in locomotion while the upper body attacks.

3. Attach the Action to the Character

The saved Action Clip must be attached to the character asset once. This lets the runtime merge the source animation onto the character skeleton and register the Action Name for scripts.

1. **Open the character asset**
   Double-click Wizard.3dgcharacter or open it from Character Editor.
2. **Select Animation**
   Keep the locomotion Animation Graph assigned.
3. **Find Standalone Action Clips**
   Open the separate action list below the graph settings.
4. **Add StaffAttack.3dgclip**
   Use the searchable action dropdown and attach the saved asset.
5. **Save the character**
   The character now carries the standalone action reference.
6. **Update the scene instance**
   Add the saved character to the scene or apply the edited character to its linked instance.

Important name rule
The script uses the Action Name stored inside the .3dgclip. The match is exact and case-sensitive: **StaffAttack** is different from **staffattack**.

Quick preview checks

- The Character Editor preview displays the correct skeletal model.
- The locomotion graph still previews idle, walk and run normally.
- The attached action appears under Standalone Action Clips.
- The listed body mask and fade values match the Clip Editor settings.
- The scene character has Script Enabled available in the Inspector.

How the runtime resolves it

.3dgclip	Character	Runtime	Script
source + action settings	attaches the action	merges animation by skeleton	PlayActionClip(name)

4. Create the Non-Spammable Attack Script

Select the player character in the scene. In the Inspector, open the script section, choose the **Empty** template, enter **PlayerAttack**, and click **Create Script**. Replace the generated class with the following code.

Content/Scripts/PlayerAttack.h

```
#pragma once

#include <engine/gameplay/Script.h>
#include <GLFW/glfw3.h>

#include <algorithm>

class PlayerAttack final : public engine::Script {
public:
    void OnUpdate(float dt) override {
        m_recoveryRemaining =
            std::max(0.0f, m_recoveryRemaining - dt);

        const bool attackPressed =
            WasMouseButtonPressed(GLFW_MOUSE_BUTTON_LEFT) ||
            WasKeyPressed(GLFW_KEY_F);

        if (!attackPressed) {
            return;
        }

        // Gate 1: never restart an action that is still playing.
        if (IsAnimationActionPlaying()) {
            return;
        }

        // Gate 2: add a short recovery after an accepted attack.
        if (m_recoveryRemaining > 0.0f) {
            return;
        }

        // Start recovery only when the named action was found and played.
        if (PlayActionClip("StaffAttack")) {
            m_recoveryRemaining = 0.65f;
        }
    }

private:
    float m_recoveryRemaining = 0.0f;
};
```

Why this cannot be spammed
Was...Pressed fires once per press, not every frame while held. **IsAnimationActionPlaying()** prevents the current attack from restarting. The recovery timer prevents another attack for 0.65 seconds after a successful start.

5. Attach, Compile and Test

1. **Attach the script**
   Select the character, choose PlayerAttack from Saved Script, then click Attach Selected.
2. **Enable it**
   Confirm Script Enabled is checked and Script Class is PlayerAttack.
3. **Save the scene**
   Save before compiling so the editor restores the same setup.
4. **Compile**
   Use Compile Scripts & Restart. The editor registers generated scripts for editor Play and project builds.
5. **Play**
   Press left mouse or F. The attack should start once. Holding the control should not repeat it.
6. **Stress-test spam prevention**
   Click rapidly during the action and during recovery. No restart should occur.

Expected behavior

Test	Expected result
Tap attack once	One complete StaffAttack action plays.
Hold attack	No repeated attacks; pressed input is edge-triggered.
Click rapidly during action	Current action is not restarted.
Click during recovery	Input is ignored until the timer reaches zero.
Full Body action	Character movement is locked until the action completes.
Spine-masked action	Locomotion may continue while the upper body attacks.

Optional: only allow attacks while grounded

If your locomotion graph exposes an **IsGrounded** bool, add this check before PlayActionClip:

```
if (!GetAnimationBool("IsGrounded", true)) {
    return;
}
```

6. Tuning and Troubleshooting

Problem	Check
Nothing plays	Confirm StaffAttack.3dgclip is marked Action Clip, attached to the character, and saved. Check exact Action Name spelling.
Attack restarts repeatedly	Use WasKeyPressed / WasMouseButtonPressed, not IsKeyDown. Keep the IsAnimationActionPlaying gate.
Attack can still fire too quickly	Increase m_recoveryRemaining. Start around the full animation duration plus the desired recovery window.
Character moves during attack	Use Full Body by clearing the Body Mask. A masked action intentionally permits locomotion.
Character slides or lunges	Enable Strip Root Motion for an in-place attack, or author intentional gameplay movement separately.
Wrong animation plays	Reopen the .3dgclip and choose the correct internal source clip.
Compilation fails on GLFW names	Keep #include <GLFW/glfw3.h> in the script.
Changes do not appear in Play	Save the script, use Compile Scripts & Restart, then reopen Play mode.

Final checklist

- The attack FBX shares the character skeleton and bone names.
- StaffAttack.3dgclip is saved under Content and marked Action Clip.
- The action is attached under Character Editor - Standalone Action Clips.
- The script calls PlayActionClip("StaffAttack") using the exact Action Name.
- Script Enabled is checked and the PlayerAttack class is attached.
- Compile Scripts & Restart completed successfully.
- Holding or rapidly pressing attack does not restart the current action.

Ready to extend
Once this base works, add an animation event at the impact frame and use WasAnimationEvent("Hit") to apply damage at the correct moment. Keep damage separate from the input press so visuals and gameplay remain synchronized.